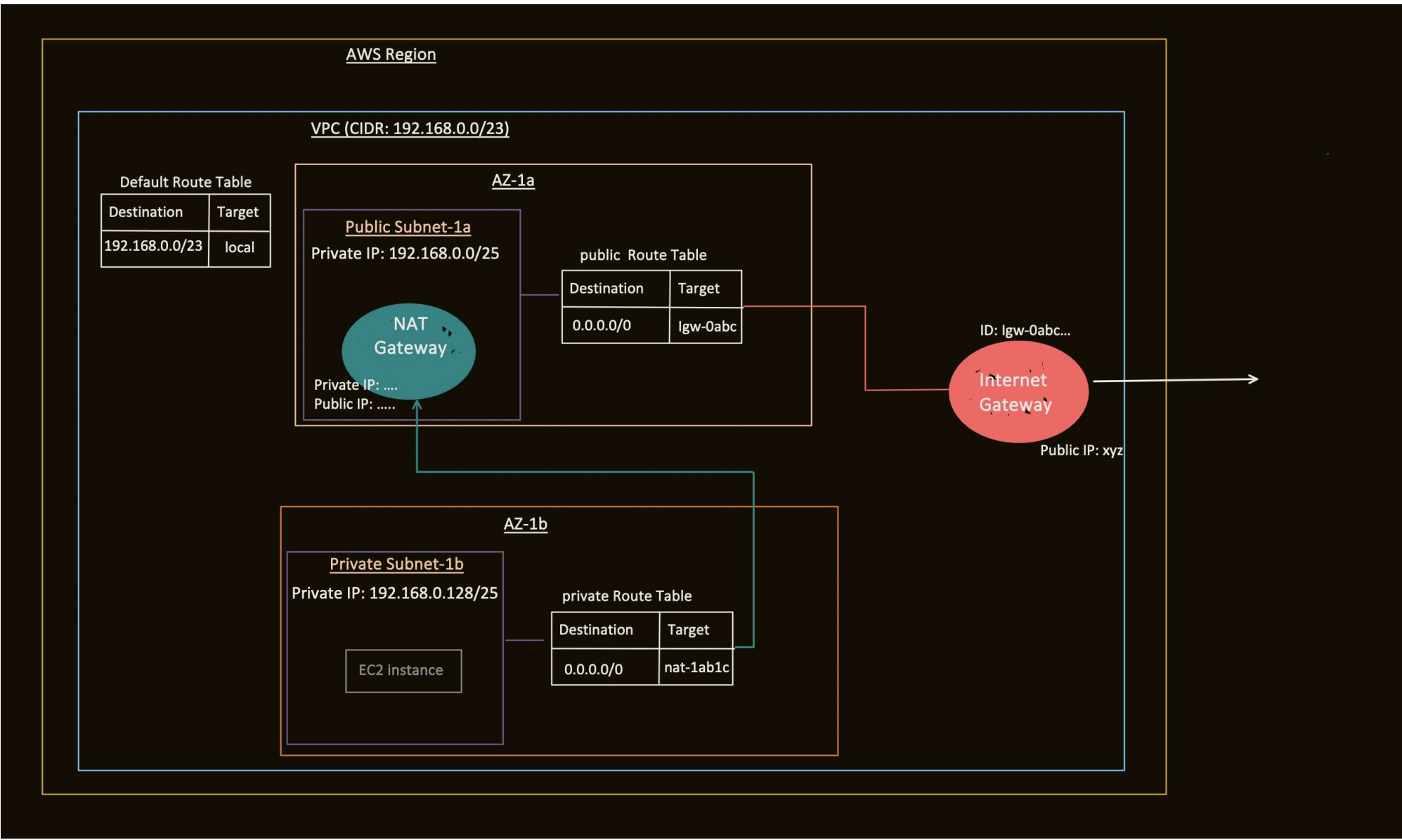


Now, lets write Terraform files for below Infrastructure.



Technically, we can define all Terraform blocks in a single .tf file.

However, for better organization and maintainability, it's recommended to split the configuration across multiple .tf files based on their purpose.

1. Setting up Terraform

provider.tf

```
terraform {
  required_version = ">= 1.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

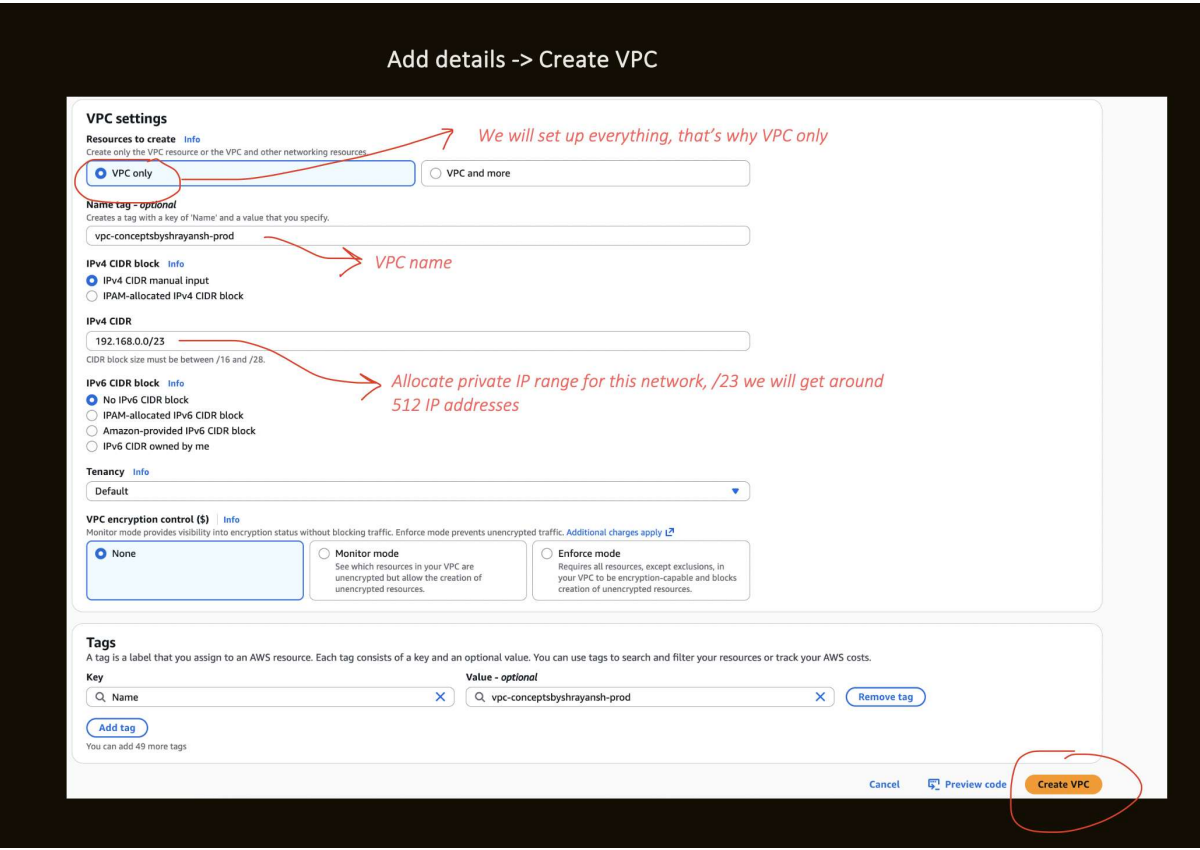
provider "aws" {
  region = var.aws_region
}
```

variable.tf

```
variable "aws_region" {
  type    = string
  default = "ap-south-1"
}
```

2. VPC

This is what we have done, while setting up the VPC manually



vpc.tf

```
resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr

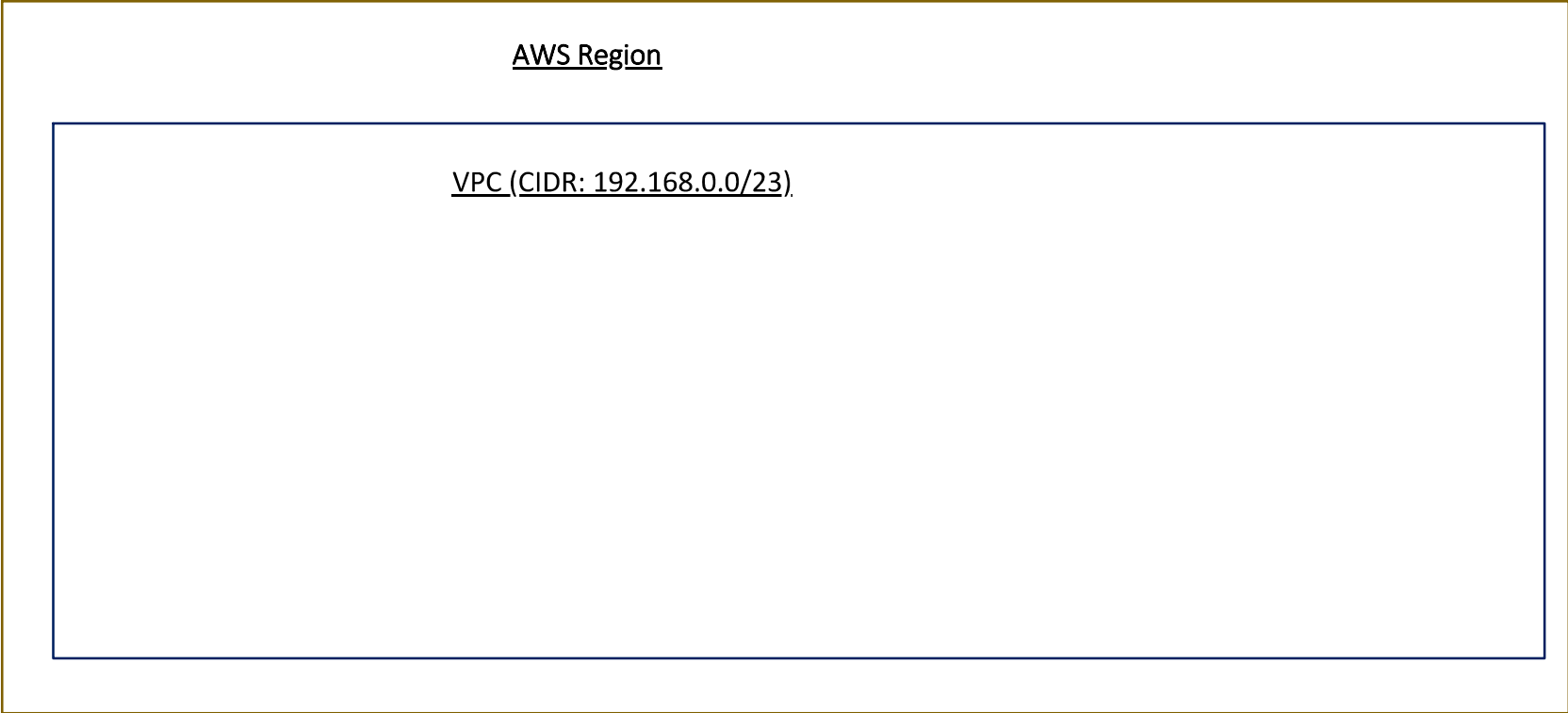
  tags = {
    Name = "vpc-${var.project_name}-prod"
  }
}
```

variable.tf

```
variable "aws_region" {
  type    = string
  default = "ap-south-1"
}

variable "project_name" {
  type    = string
  default = "conceptsbyshrayansh"
}

variable "vpc_cidr" {
  type    = string
  default = "192.168.0.0/23"
}
```



3. Subnet

For public subnet, we did below:

Create subnet

VPC

VPC ID

vpw-0a2880ac758779805 (vpc-conceptshyhrayansh-prod)

Associated VPC CIDRs

192.168.0.0/23

Subnet settings

Specify the CIDR block and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name

public-subnet-1a

Availability Zone

Asia Pacific (Mumbai) / ap-south-1a

IPv4 VPC CIDR block

192.168.0.0/23

IPv4 subnet CIDR block

192.168.0.0/25

Tags - optional

Key

Value - optional

public-subnet-1a

ADD NEW TAG

REMOVE

ADD NEW SUBNET

CANCEL

CREATE SUBNET

subnet.tf

```
# Public Subnet in AZ-1a
resource "aws_subnet" "public_1a" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.public_subnet_cidr
  availability_zone  = var.az_1a
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.project_name}-public-subnet-1a"
  }
}

# Private Subnet in AZ-1b
resource "aws_subnet" "private_1b" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.private_subnet_cidr
  availability_zone  = var.az_1b
  map_public_ip_on_launch = false

  tags = {
    Name = "${var.project_name}-private-subnet-1b"
  }
}
```

variable.tf

```
variable "aws_region" {
  type    = string
  default = "ap-south-1"
}

variable "project_name" {
  type    = string
  default = "conceptshyhrayansh"
}

variable "vpc_cidr" {
  type    = string
  default = "192.168.0.0/23"
}

variable "public_subnet_cidr" {
  type    = string
  default = "192.168.0.0/25"
}

variable "private_subnet_cidr" {
  type    = string
  default = "192.168.0.128/25"
}

variable "az_1a" {
  type    = string
  default = "ap-south-1a"
}

variable "az_1b" {
  type    = string
  default = "ap-south-1b"
}
```

4. Internet Gateway

If did below manually:

Name -> create Internet Gateway

Create internet gateway

Internet gateway settings

Tags - optional

ADD NEW TAG

CREATE INTERNET GATEWAY

Our newly created Internet Gateway -> Attach to VPC

igw-0f46d967b6d8d807 / conceptshyhrayansh-internetgateway

Attach to VPC

Attach to VPC

Manage tags

Delete

Tags (1)

NAME

conceptshyhrayansh-internetgateway

Select our VPC and Click Attach

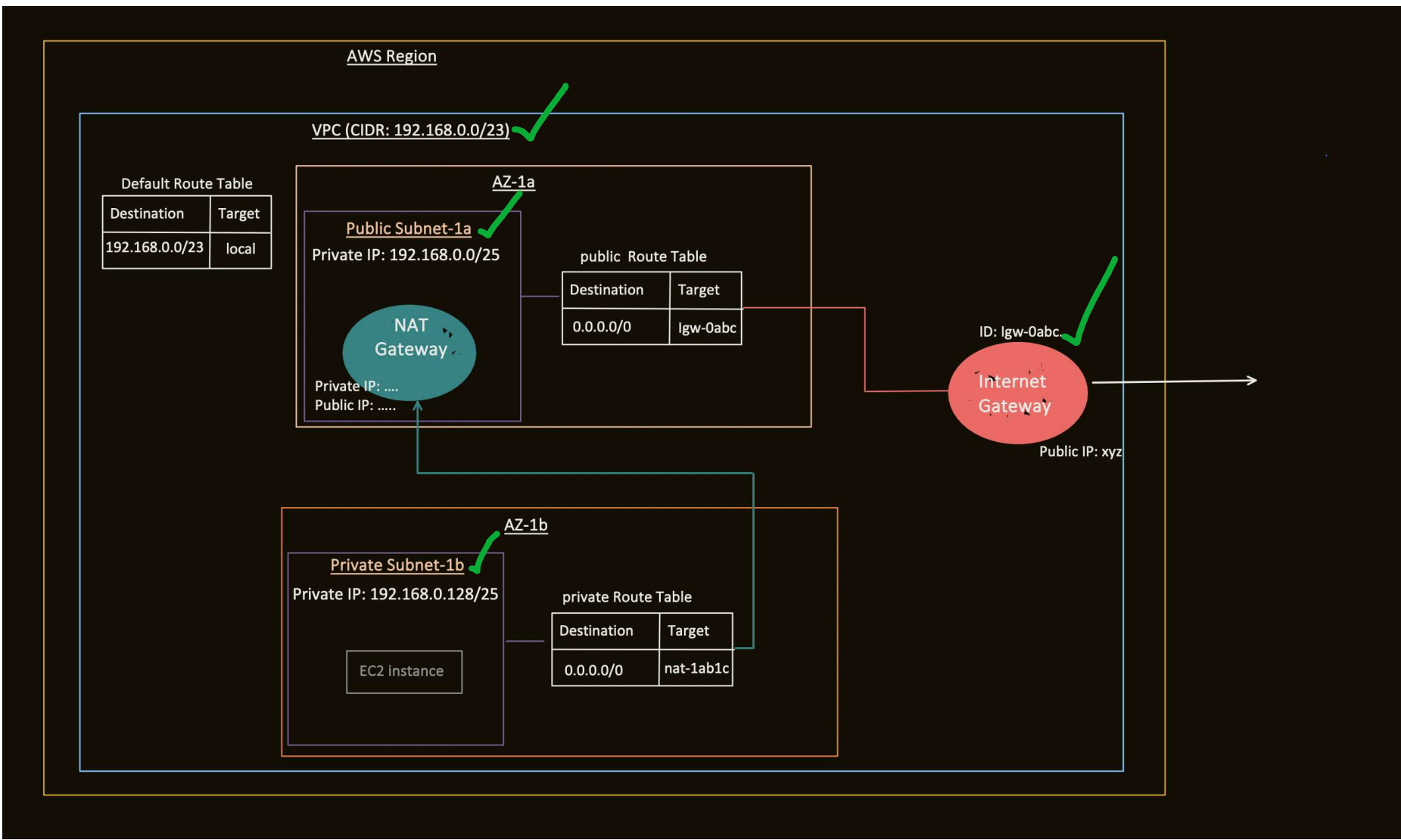
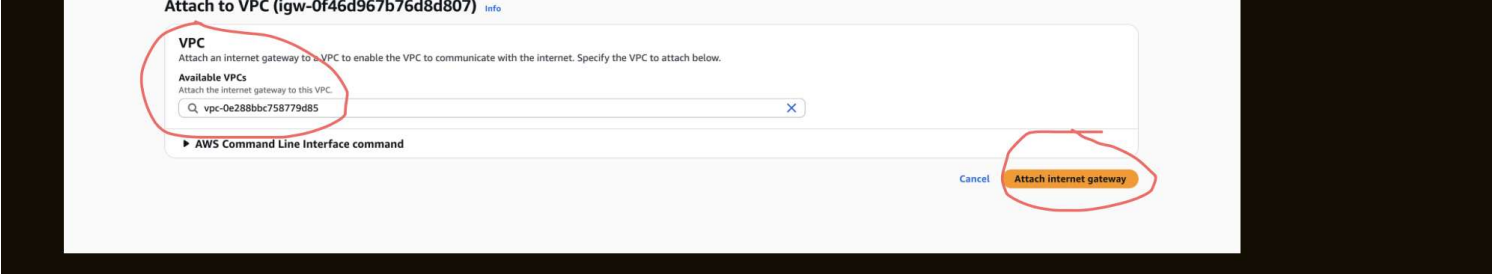
igw-0f46d967b6d8d807

Attach to VPC

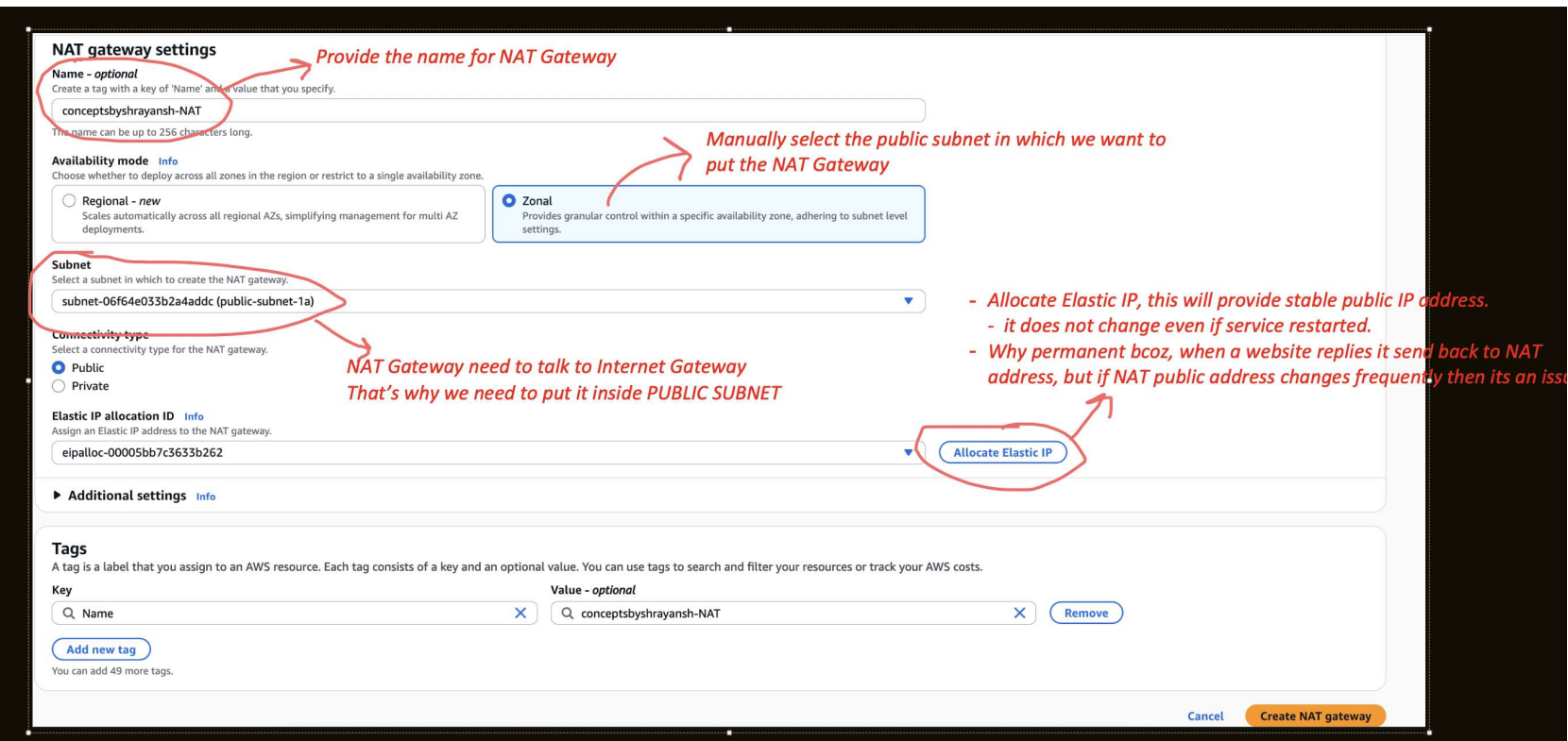
Internet-gateway.tf

```
resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "${var.project_name}-igw"
  }
}
```

5. NAT Gateway



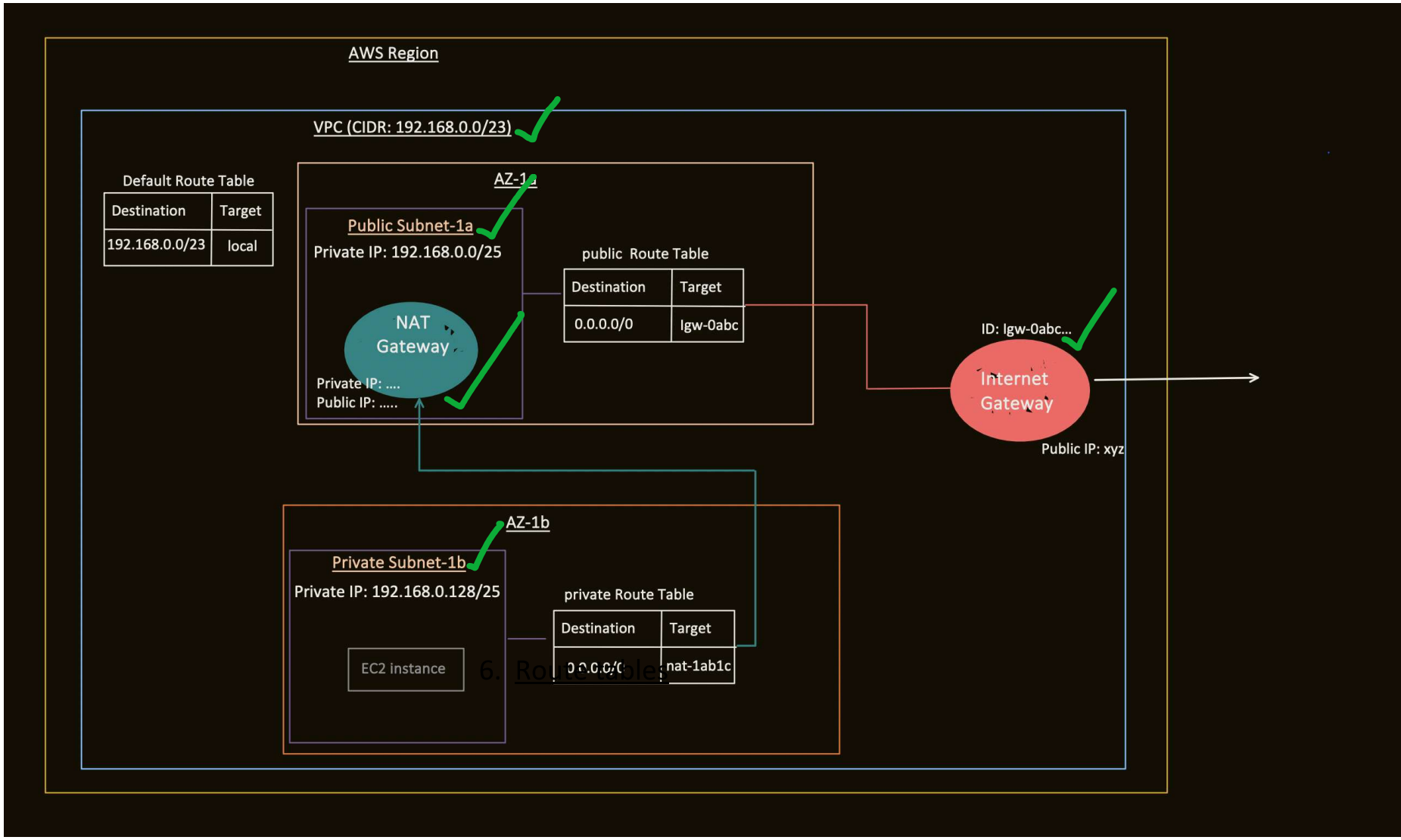
nat-gateway.tf

```
#Elastic IP for the NAT Gateway
resource "aws_eip" "nat" {
  domain = "vpc"
}

resource "aws_nat_gateway" "main" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public_1a.id

  tags = {
    Name = "${var.project_name}-nat-gateway"
  }

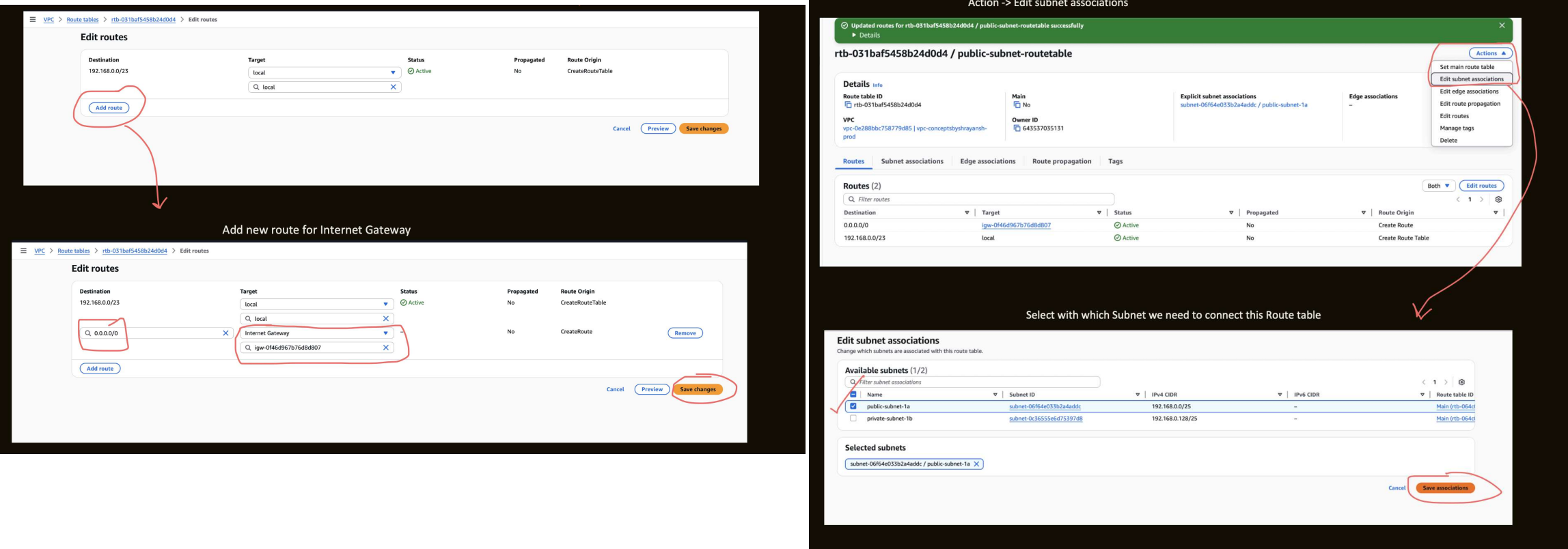
  # Must have Internet Gateway before creating NAT Gateway
  depends_on = [aws_internet_gateway.main]
}
```



6. Route Tables

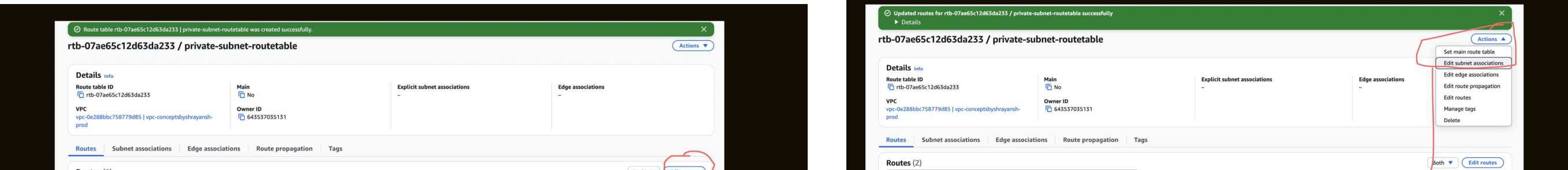
Public Route table:

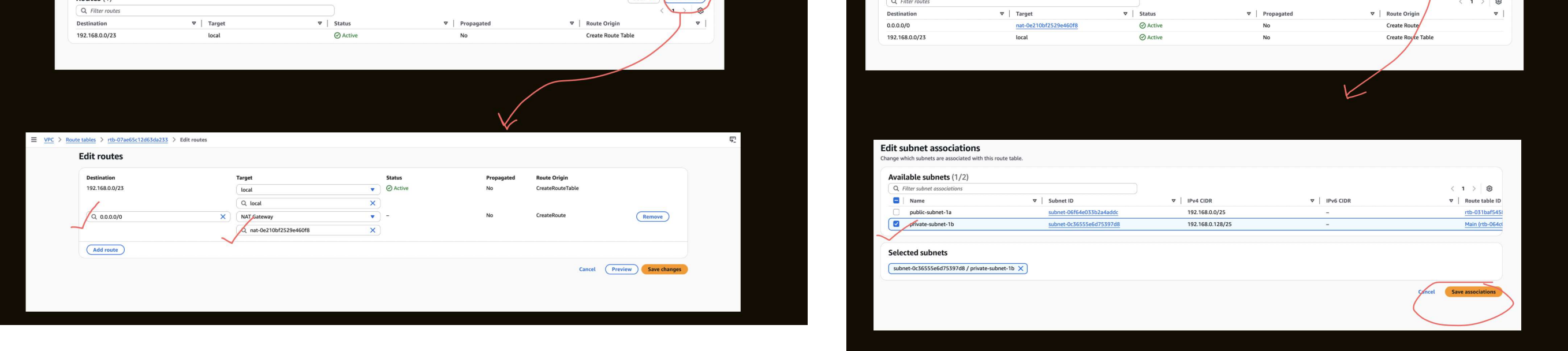
1. Add internet gateway route
2. Attach to public subnet



Private Route table:

1. Add NAT gateway route
2. Attach to private subnet





Route-tables.tf

```
# PUBLIC ROUTE TABLE
# Routes internet traffic (0.0.0.0/0) to Internet Gateway

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
}

resource "aws_route" "public_internet_access" {
  route_table_id = aws_route_table.public.id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id = aws_internet_gateway.main.id
}

resource "aws_route_table_association" "public_1a" {
  subnet_id = aws_subnet.public_1a.id
  route_table_id = aws_route_table.public.id
}

# PRIVATE ROUTE TABLE
# Routes internet traffic (0.0.0.0/0) to NAT Gateway

resource "aws_route_table" "private" {
  vpc_id = aws_vpc.main.id
}

resource "aws_route" "private_internet_access" {
  route_table_id = aws_route_table.private.id
  destination_cidr_block = "0.0.0.0/0"
  nat_gateway_id = aws_nat_gateway.main.id
}

resource "aws_route_table_association" "private_1b" {
  subnet_id = aws_subnet.private_1b.id
  route_table_id = aws_route_table.private.id
}

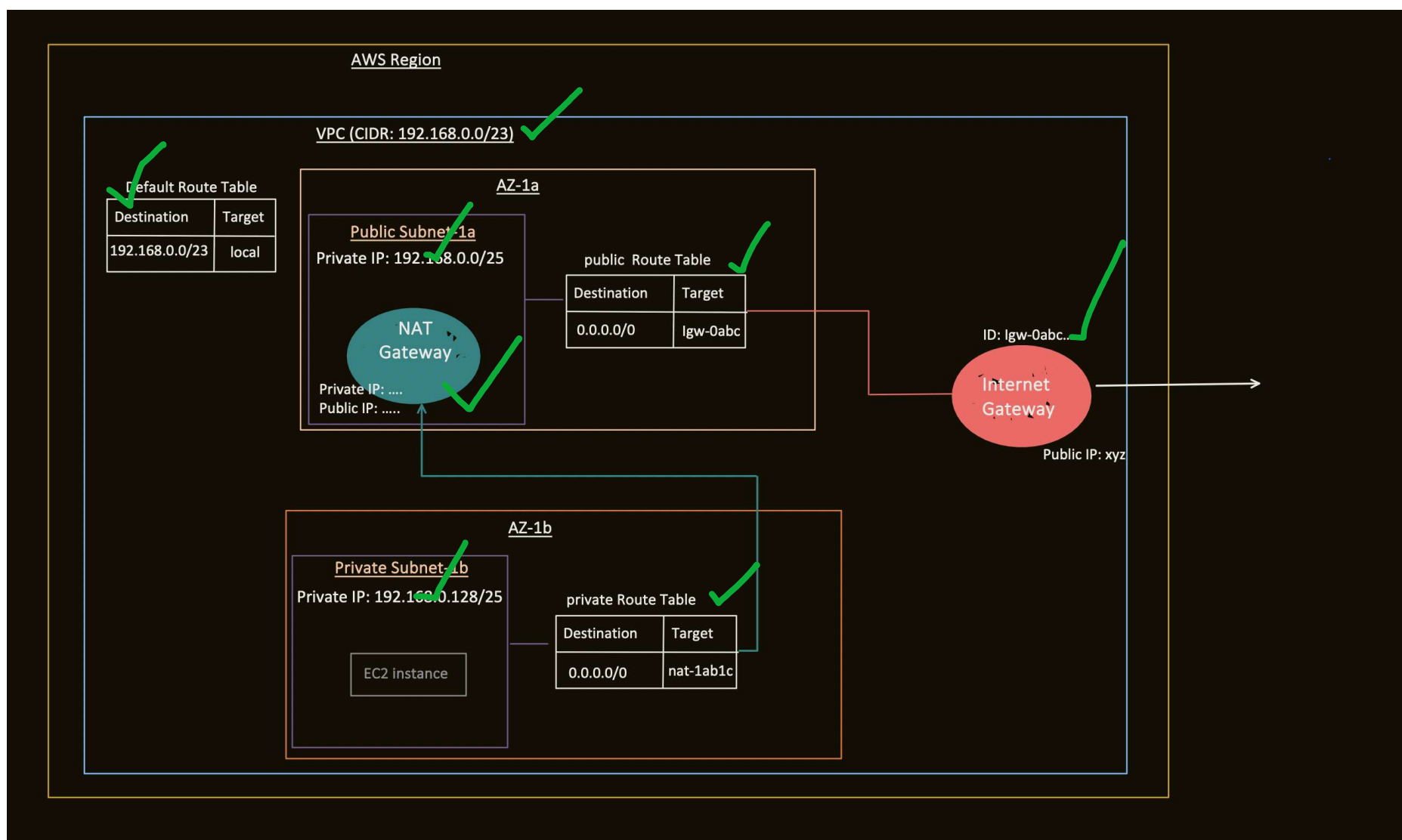
# DEFAULT ROUTE TABLE (VPC's Main Route Table)

resource "aws_default_route_table" "default" {
  default_route_table_id = aws_vpc.main.default_route_table_id
}
```

aws_route_table:
To create Route table

aws_route:
Represent entry within a route table

aws_route_table_association:
Subnet and route table association



All done, now how to execute this .tf files?

1. Install terraform (pre-requisite)

macOS:

brew tap hashicorp/tap
brew install hashicorp/tap/terraform

On Windows: Download from: <https://www.terraform.io/downloads>

Will install the latest stable terraform version.

2. Set AWS access key and secret in environment variable

export AWS_ACCESS_KEY_ID=AKIAZL...
export AWS_SECRET_ACCESS_KEY=xXmOet...

3. Initialize Terraform

terraform init

What this does:

- Validate terraform version
- Downloads AWS provider plugin
- Etc.

4. Review plan (Optional)

terraform plan

5. Apply the configurations

terraform apply

When we are done and want to delete everything:

terraform destroy